

数字系统设计

Digital System Design

-----微处理器设计2

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信网络工程研究所

Zhejiang University

Spring 2026 – Lec. #13

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室（可以钉钉、微信或邮件联系）.

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**
 - 期中考试 **10%**
 - Homework Sets **15% (12~20)%**
 - Project **5% (5~7)%**
 - Class Room Check+小测验等 **10% (7~10)%**
 - 作业每周一上交, **截止期为课后一周内**有效。
- **Project**
 - Project-1必做 (1 members)
 - Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)
- **Finial Exam**期末闭卷考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

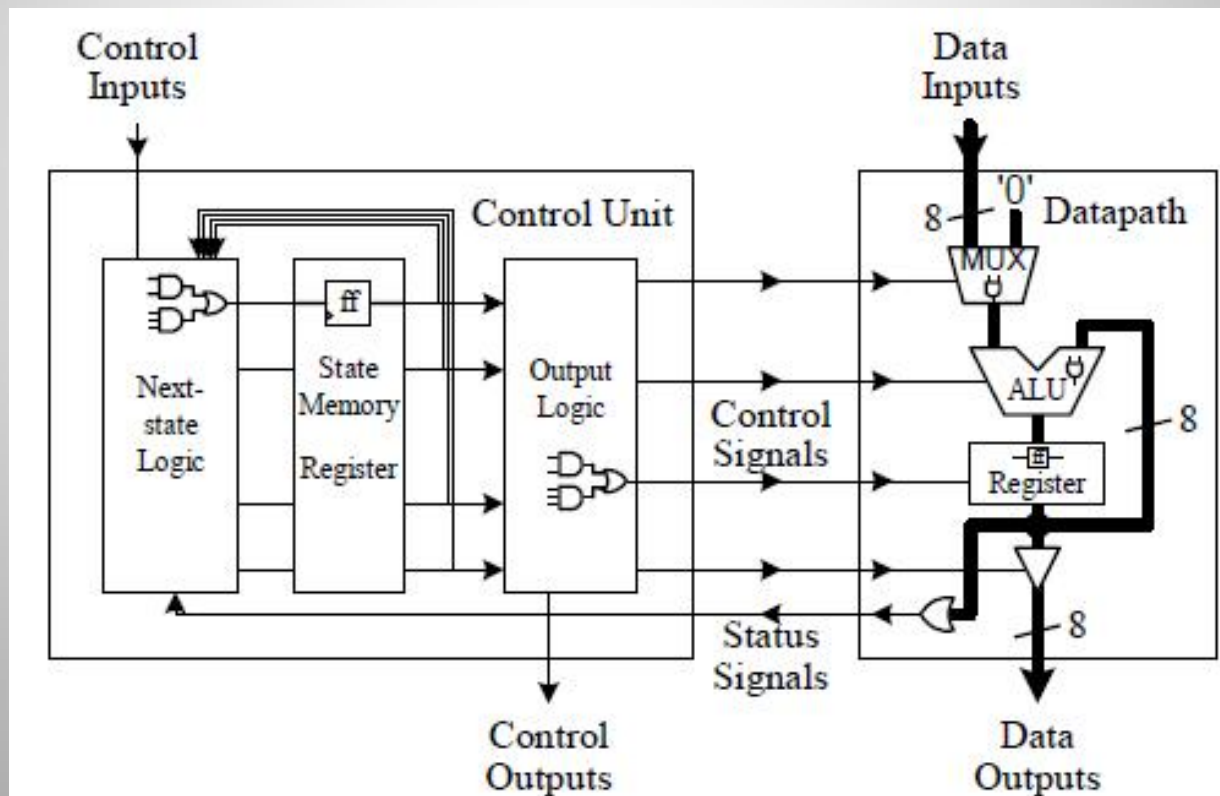
- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

讲义第十章 微处理器设计

-----硬连线型微处理器

微处理器内部结构

- 微处理器的两个主要部分：
 - 控制单元
 - » 用于控制数据通路的所有操作，实现微处理器运算的正确性
 - 数据通路
 - » 主要包括运算单元ALU、存储单元(寄存器)及其相互连接



微处理器内部结构

微处理器的三个执行步骤

- 指令周期

- (1)取指

- » 即从程序存储器中把PC所指向的指令取出, 拷贝到IR;
 - » PC+1指向下一条指令的地址;

- (2)译指

- » 从IR中提取出操作码, 对指令所要做的操作进行翻译;
 - » 决定当前执行那一条指令, 跳转到那一个对应的状态去;

- (3)执行指令

- 每一步将在有限状态机的一个状态被执行。
 - 每个指令通常在一个时钟周期内执行
 - 有些情况下一些存储器操作的指令可能需要两个甚至更长的时钟周期, 这种情况下就需要更多的状态来实现正确的时序。

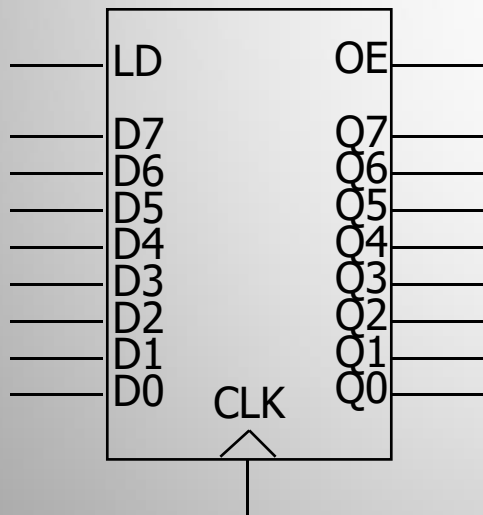
- 一般而言, 程序的指令都是存储在外部寄存器中的, 所以除了CPU外, 微处理器一般还需要外部存储器以及连接外部存储器与CPU的地址以及数据总线。

mips指令格式

- **1) R型-----三个寄存器操作数格式**
 - 用于如`add`和`sub`指令, 有三个寄存器操作数。
- **2) I型-----两个寄存器操作数格式**
 - 用于如`lw`和`sw`指令, 具有两个寄存器操作数和一个16位立即数。
- **3) J型-----无寄存器格式**
 - 有一个26位的立即数, 无寄存器。
- MIPS采用32位指令

Registers

- Selectively loaded – EN or LD input
- Output enable – OE input
- Multiple registers – group 4 or 8 in parallel

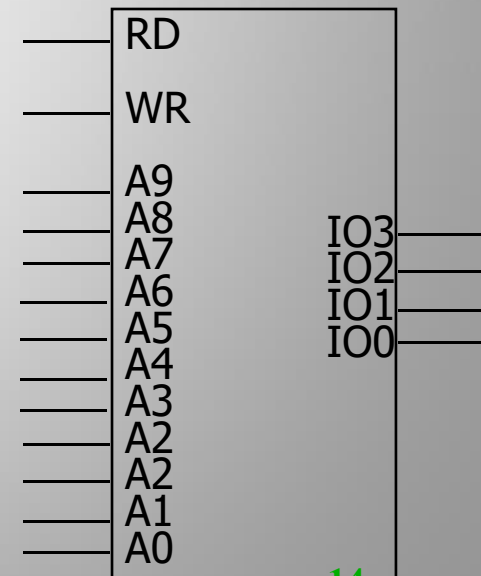


OE asserted causes FF state to be connected to output pins; otherwise they are left unconnected (high impedance)

LD asserted during a lo-to-hi clock transition loads new data into FFs

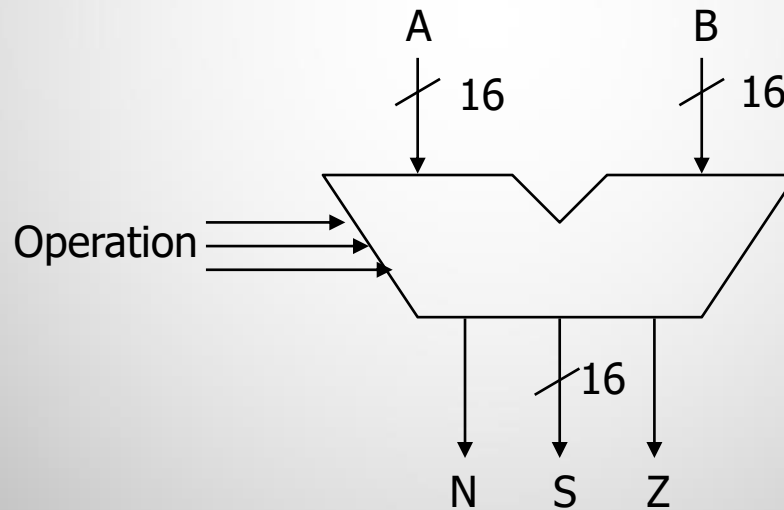
Memories

- Larger Collections of Storage Elements
 - Implemented not as FFs but as much more efficient latches
 - High-density memories use 1-5 switches (transistors) per bit
- Static RAM – 1024 words each 4 bits wide
 - Once written, memory holds forever (not true for denser dynamic RAM)
 - Address lines to select word (10 lines for 1024 words)
 - Read enable
 - » Same as output enable
 - » Often called chip select
 - » Permits connection of many chips into larger array
 - Write enable (same as load enable)
 - Bi-directional data lines
 - » output when reading, input when writing



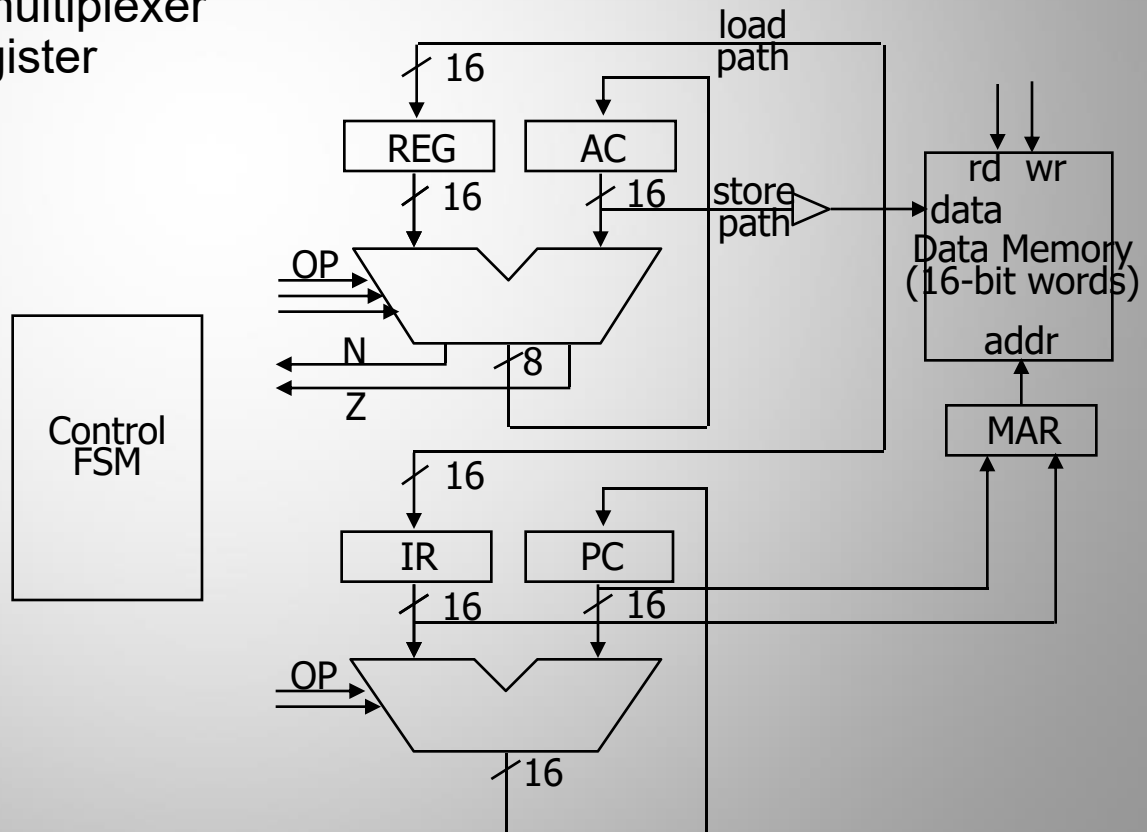
Data Path (ALU)

- ALU Block Diagram
 - Input: data and operation to perform
 - Output: result of operation and status information



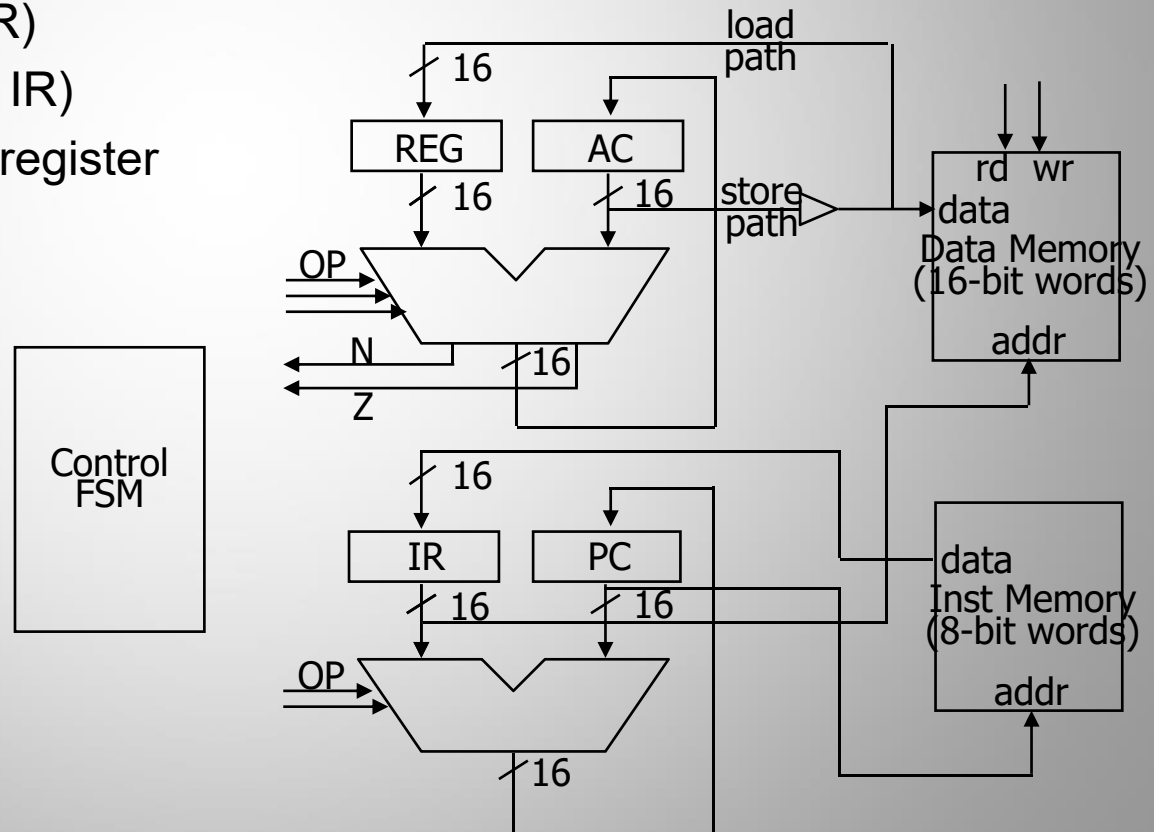
Block Diagram of Processor

- Register Transfer View of **Princeton** Architecture
 - Which register outputs are connected to which register inputs
 - Arrows represent data-flow, other are control signals from control FSM
 - MAR may be a simple multiplexer rather than separate register
 - MBR is split in two (REG and IR)
 - Load control for each register



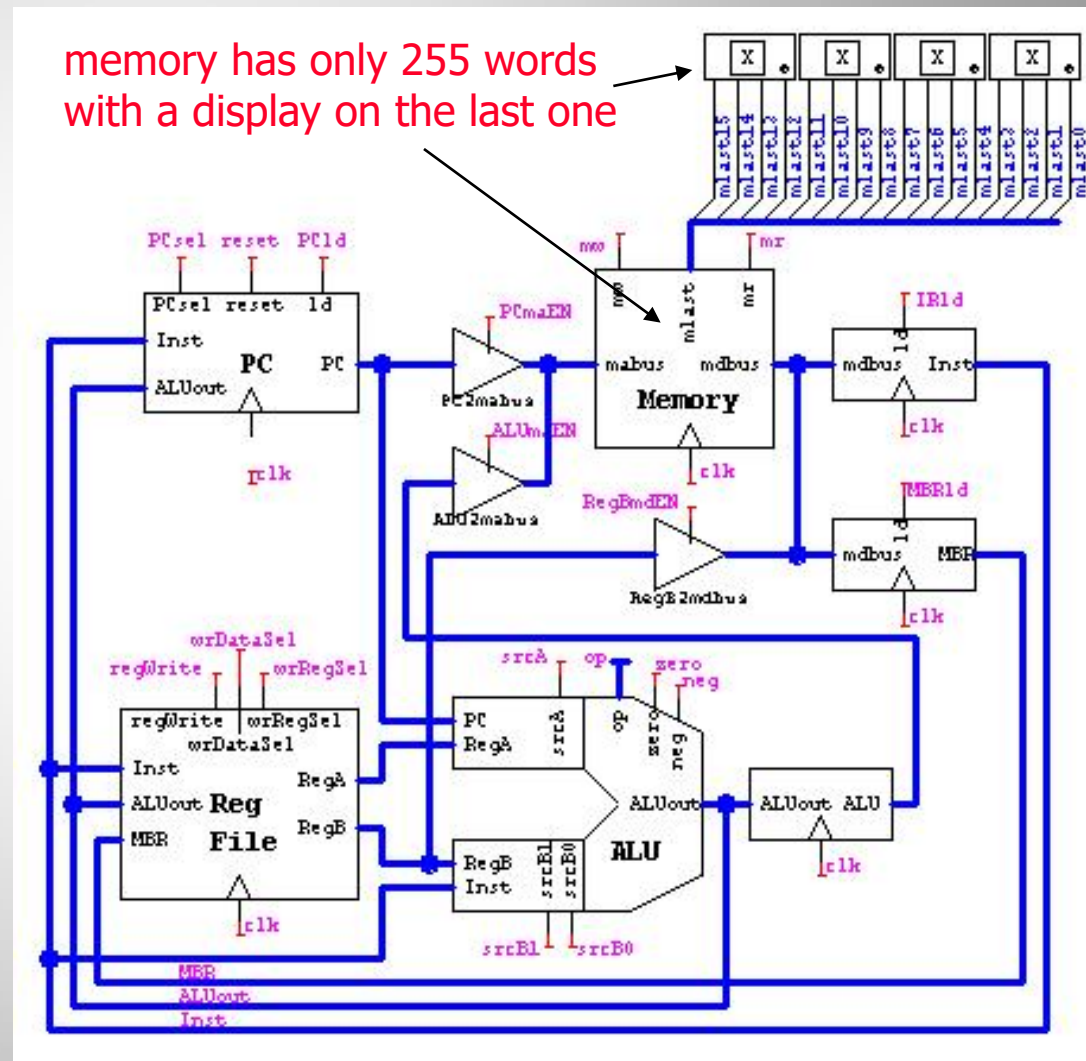
Block Diagram of Processor

- Register transfer view of **Harvard** architecture
 - Which register outputs are connected to which register inputs
 - Arrows represent data-flow, other are control signals from control FSM
 - Two MARs (PC and IR)
 - Two MBRs (REG and IR)
 - Load control for each register

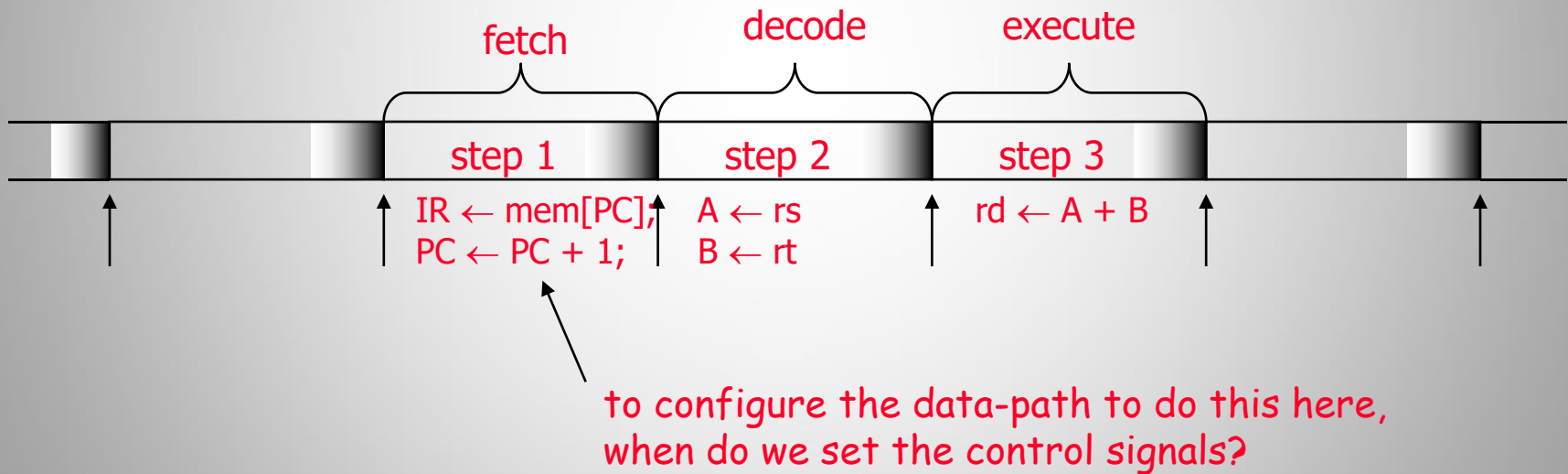


A Simplified Processor Data-path and Memory

- Princeton architecture
- Register file
- Instruction register
- PC incremented through ALU
- Modeled after MIPS rt000 (used in 61C textbook by Patterson & Hennessy)
 - Really a 32 bit machine
 - We'll do a 16 bit version



Review of FSM Timing



FSM Controller for CPU

- Now we need to repeat this for all the instructions of our processor
 - Fetch and decode states stay the same
 - Different execution states for each instruction
 - » Some may require multiple states if available register transfer paths require sequencing of steps

Alternative Ways to Implement Processor FSMs

- "Random Logic" based on Moore and Mealy Design
 - Classical Finite State Machine Design
- Divide and Conquer Approach: Time-State Method
 - Partition FSM into multiple communicating FSMs
- Exploit Logic Block Functionality: Jump Counters
 - Counters, Multiplexers, Decoders
- Microprogramming: ROM-based methods
 - Direct encoding of next states and outputs

10.3 微处理器设计示例EC-1

• 指令集-1

- 五个8bits指令
- Opcode=3bits, 000, 001和010三个编码没有定义, nop

Instruction	Encoding	Operation	Comment
IN A	011 xxxxx	$A \leftarrow \text{input}$	Input to A
OUT A	100 xxxxx	$\text{output} \leftarrow A$	Output from A
DEC A	101 xxxxx	$A \leftarrow A - 1$	Decrement A
JNZ address	110 xaaaa	if ($A \neq 0$) then $PC = \text{aaaa}$	Jump to address if A is not zero
HALT	111 xxxxx	Halt	Halt execution

Notations:

A = accumulator.

PC = program counter.

aaaa = four bits for specifying a memory address.

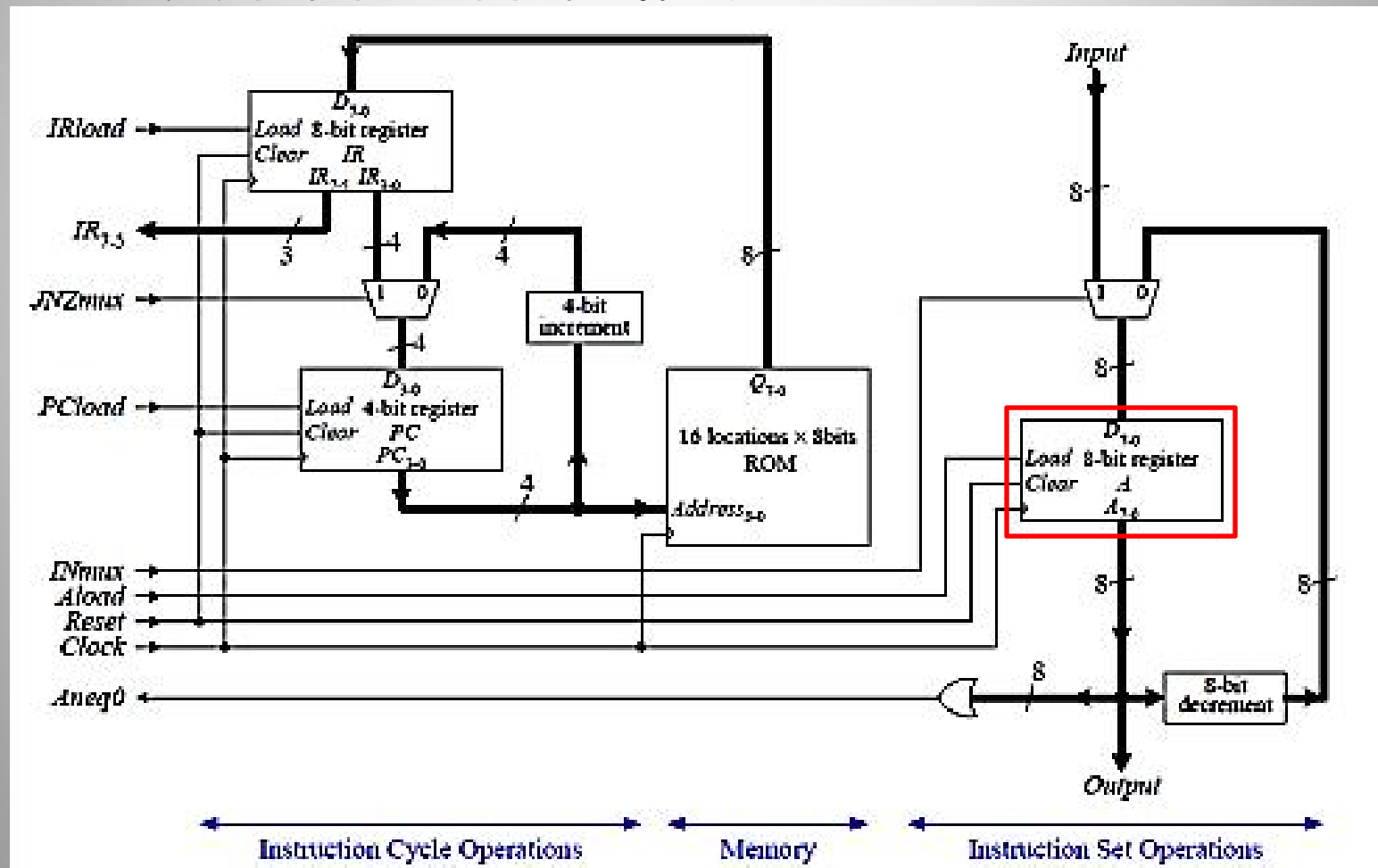
x = don't cares.

- 1) 指令IN A输入一个8比特的数值，并存储到累加器A中；
- 2) 指令OUT A将累加器A中的数值拷贝到输出端口；
- 3) 指令DEC A把累加器A中的数值减1后再存入到累加器A中；
- 4) 指令JNZ（Jump Not Zero）测试A中的数值是否为0，假如为0，这个指令什么也不做，假如不为0，则将指令中的后四位，即aaaa（表示跳转地址）加载到寄存器PC中；
- 5) 指令HALT通过使CPU停止在halt状态从而停止微处理器的运行，直到系统复位跳出halt状态。

数据通路-1

指令集中操作的数据通路

- (1) 执行指令周期中的取指操作，并递增PC值或者加载新的PC值；
- (2) 存储器；
- (3) 实现指令集中所有指令的操作。

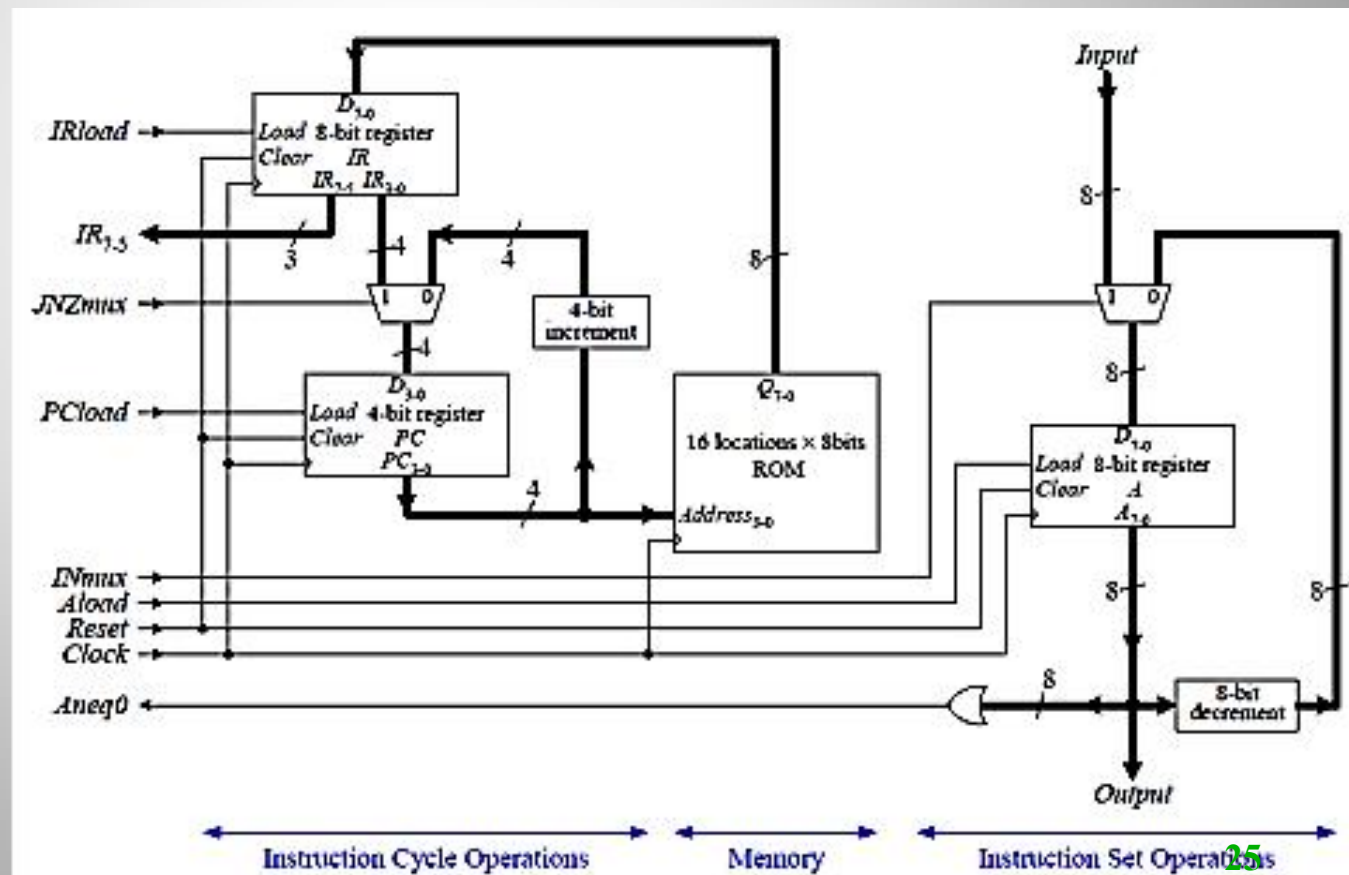


数据通路-1

- 一个PC寄存器, 4比特, +递增单元
- 一个IR寄存器, 8比特
- 一个具有16个存储单元的8位程序存储器
- 一个8比特的累加器A, +递减单元
- 指令IN A, 累加器A的输入来自于数据输入端口;
- 指令DEC A, 累加器A的输入来自于递减单元的输出;
- 指令OUT A, 无需特别的数据通路的操作, 累加器的输出直接连接到数据输出端口;
- 指令JNZ A, 需要一个8比特的OR门连接到累加器的所有输出上, 来测试条件A是否不等于0。IR寄存器的低4位数据加载到PC寄存器中;
- 指令HALT, 不需要任何额外的操作。

控制信号

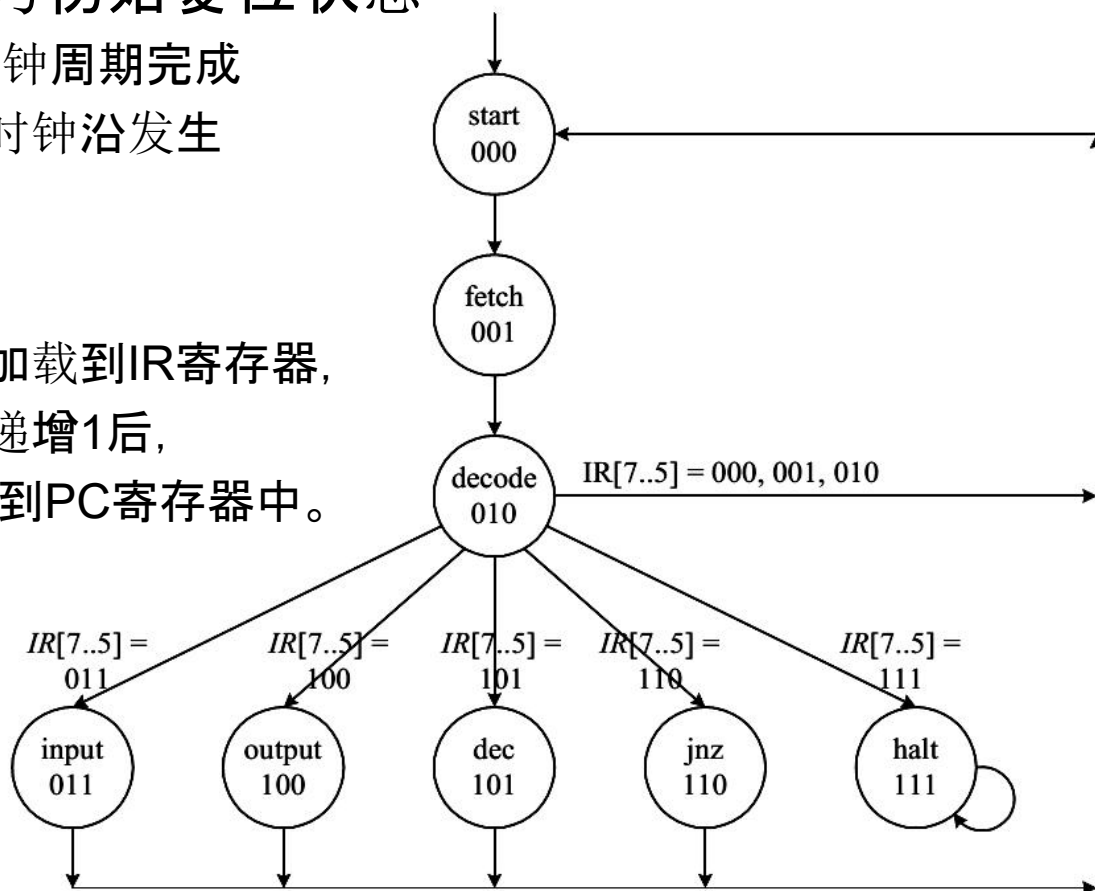
- 数据通路中的控制字主要有5个
 - 即IRload, PCLoad, INmux, Aload和JNZmux。
- 给控制单元提供一个状态反馈信号
 - Aneq0



我们的设计中，为了保持设计的简洁，我们把程序存储器跟CPU一起设计在数据通路中了

控制单元-1

- 控制单元的状态图
- 状态**START(000)**作为初始复位状态
 - 指令JNZ A需要额外的时钟周期完成
 - PC加载新地址在下一个时钟沿发生
- 在**FETCH**状态
 - 控制IRload信号有效,
 - 将PC寄存器指向的程序加载到IR寄存器,
 - 然后PC寄存器中的内容递增1后,
 - 信号PCload将新PC加载到PC寄存器中。
- 在**DECODE**状态
 - 测试IR寄存器的高三位,
 - 即IR₇₋₅解码,
 - 跳转到对应的状态。



(a)

五个指令的执行状态

- 指令IN A操作

- 设置INmux信号为1来选择输入信号为外部端口,
- 设置Aload信号为1来加载输入信号到累加器A。
- 注意, 为了输入指令能够读取到正确的输入值, 输入数据需要在微处理器复位之前就设置好。
- 而且, 由于INPUT状态不需要等待一个类似于回车键的信号, 因此, 即使有多个数据输入, 也只能有一个数据会被读入进来。

- 指令DEC A

- 需要设置INmux信号为0、Aload信号为1,
- 以使递减单元的输岀被连接到累加器并加载进来。

- 指令JNZ

- 使JNZmux信号有效以将IR寄存器中的低4位地址信号加载到PC寄存器。
- 状态反馈信号Aneq0=1时, 控制信号PCload信号被设置为有效。

- 到HALT状态

- 将会无条件的停留在HALT状态
- 微处理器看上去像停止操作一样。

状态图

- 次态表、实现表, 激励方程

Current State $Q_2Q_1Q_0$	Next State $Q_{2next}Q_{1next}Q_{0next}$ (Implementation $D_2D_1D_0$)							
	IR_7, IR_6, IR_5							
	000 nop	001 nop	010 nop	011 input	100 output	101 dec	110 jnz	111 halt
000 start	001	001	001	001	001	001	001	001
001 fetch	010	010	010	010	010	010	010	010
010 decode	000	000	000	011	100	101	110	111
011 input	000	000	000	000	000	000	000	000
100 output	000	000	000	000	000	000	000	000
101 dec	000	000	000	000	000	000	000	000
110 jnz	000	000	000	000	000	000	000	000
111 halt	111	111	111	111	111	111	111	111

(b)

$$D_2 = Q_2Q_1Q_0 + Q_2'Q_1Q_0'IR_7$$

$$D_1 = Q_2Q_1Q_0 + Q_2'Q_1Q_0'(IR_6IR_5 + IR_7IR_6) + Q_2'Q_1'Q_0$$

$$D_0 = Q_2Q_1Q_0 + Q_2'Q_1Q_0'(IR_6IR_5 + IR_7IR_5) + Q_2'Q_1'Q_0'$$

(c)

状态控制

- 输出表、输出方程

Control Word	State $Q_2Q_1Q_0$	$IRload$	$PCload$	$INmux$	$Aload$	$JNZmux$	$Halt$
0	000 start	0	0	0	0	0	0
1	001 fetch	1	1	0	0	0	0
2	010 decode	0	0	0	0	0	0
3	011 input	0	0	1	1	0	0
4	100 output	0	0	0	0	0	0
5	101 dec	0	0	0	1	0	0
6	110 jnz	0	if ($Aneq0 = 1$) then 1 else 0	0	0	1	0
7	111 halt	0	0	0	0	0	1

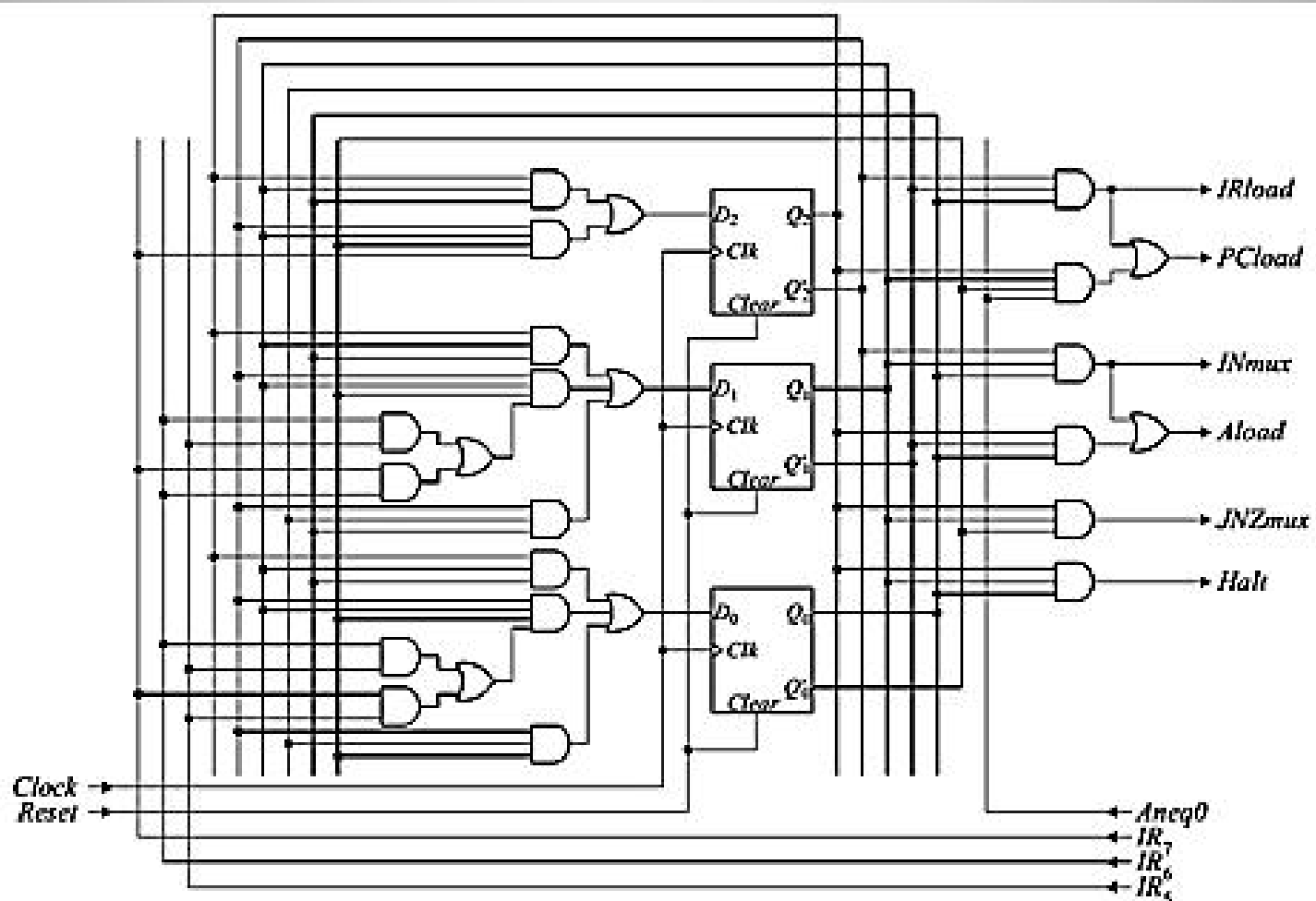
(d)

$$\begin{aligned}
 IRload &= Q_2'Q_1'Q_0 \\
 PCload &= Q_2'Q_1'Q_0 + Q_2Q_1Q_0'(Aneq0) \\
 INmux &= Q_2'Q_1Q_0 \\
 Aload &= Q_2'Q_1Q_0 + Q_2Q_1'Q_0 \\
 JNZmux &= Q_2Q_1Q_0' \\
 Halt &= Q_2Q_1Q_0
 \end{aligned}$$

(e)

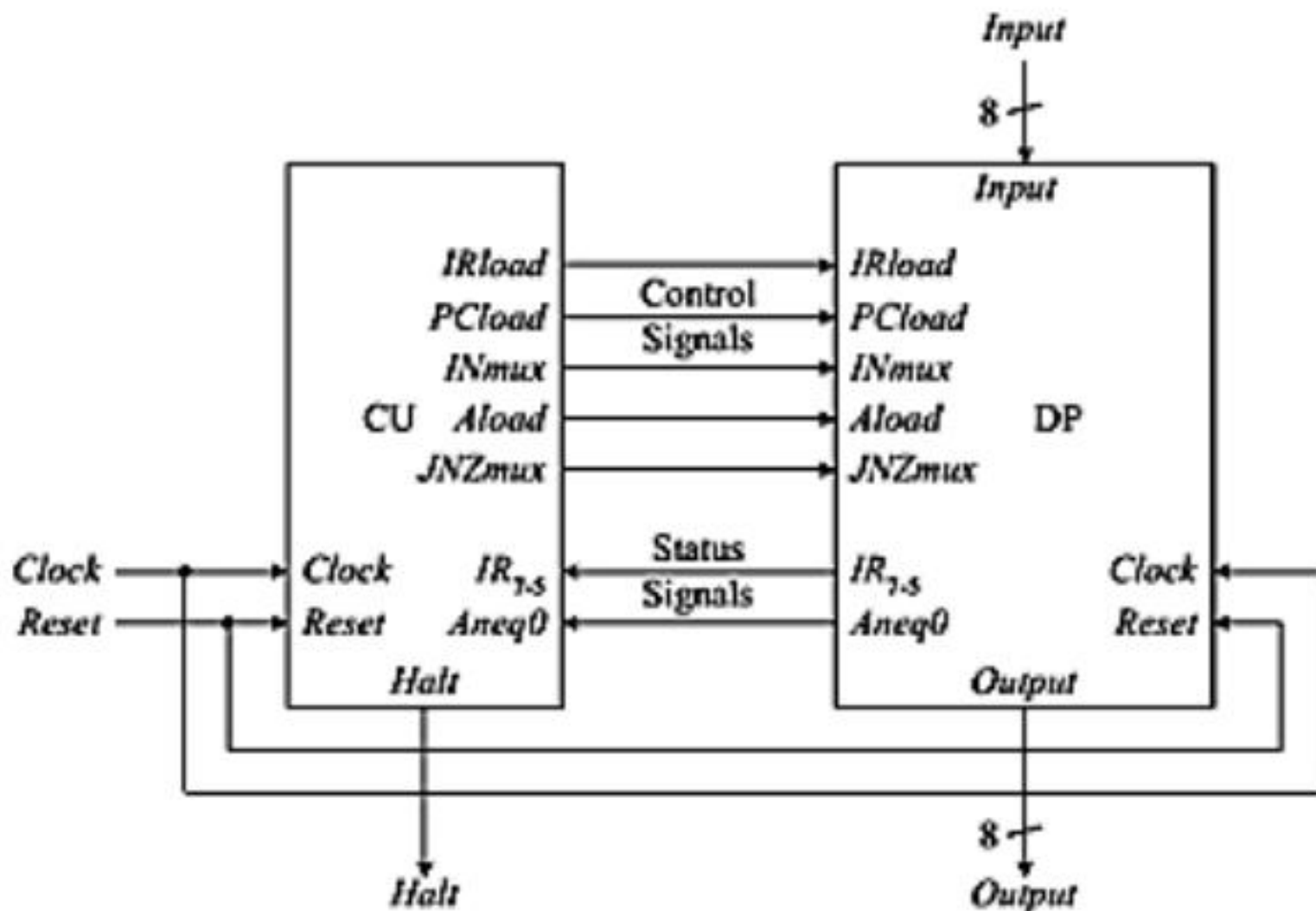
控制单元

- 完整控制电路



完整电路-1

- 数据通路电路与控制单元电路信号连接



示例汇编程序-1

程序必须在数据通路和微处理器实现之前，加载到存储器中去

• 测试程序

- 输入一个数据，将此数据进行递减直至0。

```
IN A      -- input a value into the A register
loop: OUT A  -- output the value from the A register
DEC A     -- decrement A by one
JNZ loop  -- go back to loop if A is not zero
HALT     -- halt
```

(a)

• 手工编译

Instruction	Encoding	Operation	Comment
IN A	011 xxxxx	$A \leftarrow \text{input}$	Input to A
OUT A	100 xxxxx	$\text{output} \leftarrow A$	Output from A
DEC A	101 xxxxx	$A \leftarrow A - 1$	Decrement A
JNZ address	110 xaaaa	if ($A \neq 0$) then $PC = \text{aaaa}$	Jump to address if A is not zero
HALT	111 xxxxx	Halt	Halt execution

Notations:

A = accumulator.
PC = program counter.
aaaa = four bits for specifying a memory address.
x = don't cares.

memory address	instruction encoding	
0000	01100000;	-- IN A
0001	10000000;	-- OUT A
0010	10100000;	-- DEC A
0011	11000001;	-- JNZ loop
0100	11111111;	-- HALT

(b)

指令JNZ，最右边4位表示当检测条件为真时程序跳转的地址。

假设第一个指令IN A存储在程序存储器的地址为0000，

指令JNZ将跳转到第二个指令OUT A（在程序存储器中的地址是0001）

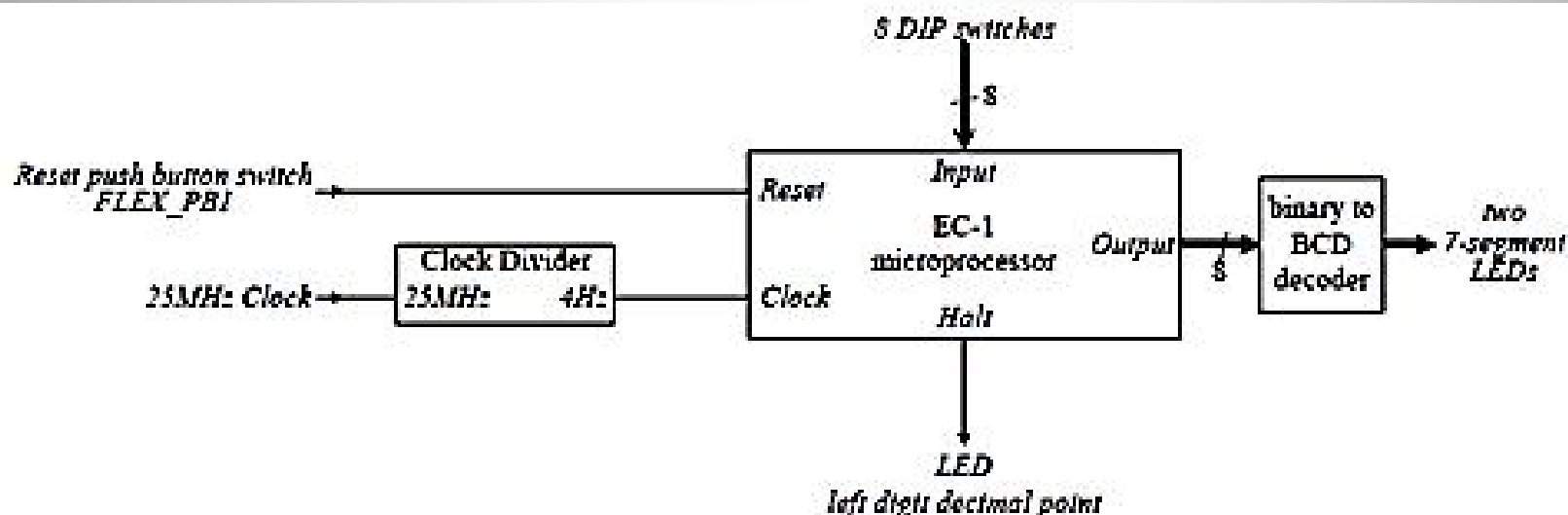
JNZ指令的最后四位操作数被设置为0001。

由于指令JNZ的操作码为110，所以综合起来指令JNZ完整的编码是11000001。

硬件电路实现-1

- 完整的计算机

- 微处理器, 存储器、输入和输出设备。
- 八个拨码开关作为输入设备, 2个七段数码管作为输出设备。
- 一个LED来显示是否已经结束工作, 一个按钮作为复位按钮。
- 较高频率(如25MHz)的时钟分频后变成4Hz, 作为微处理器的时钟。



指令集扩展---EC-2微处理器（自学）

- 指令集-2
 - 8个指令，仍是8位

Instruction	Encoding	Operation	Comment
LOAD A, address	000 aaaaa	$A \leftarrow M[aaaaa]$	Load A with content of memory location aaaaa
STORE A, address	001 aaaaa	$M[aaaaa] \leftarrow A$	Store A into memory location aaaaa
ADD A, address	010 aaaaa	$A \leftarrow A + M[aaaaa]$	Add A with M[aaaaa] and store the result back into A
SUB A, address	011 aaaaa	$A \leftarrow A - M[aaaaa]$	Subtract A with M[aaaaa] and store result back into A
IN A	100 xxxxx	$A \leftarrow \text{input}$	Input to A
JZ address	101 aaaaa	if (A = 0) then PC = aaaaa	Jump to address if A is zero
JPOS address	110 aaaaa	if (A > 0) then PC = aaaaa	Jump to address if A is positive
HALT	111 xxxxx	Halt	Halt execution

Notations:

A = accumulator.

M = memory.

PC = program counter.

aaaaa = four bits for specifying a memory address.

x = don't cares.

指令-2（自学）

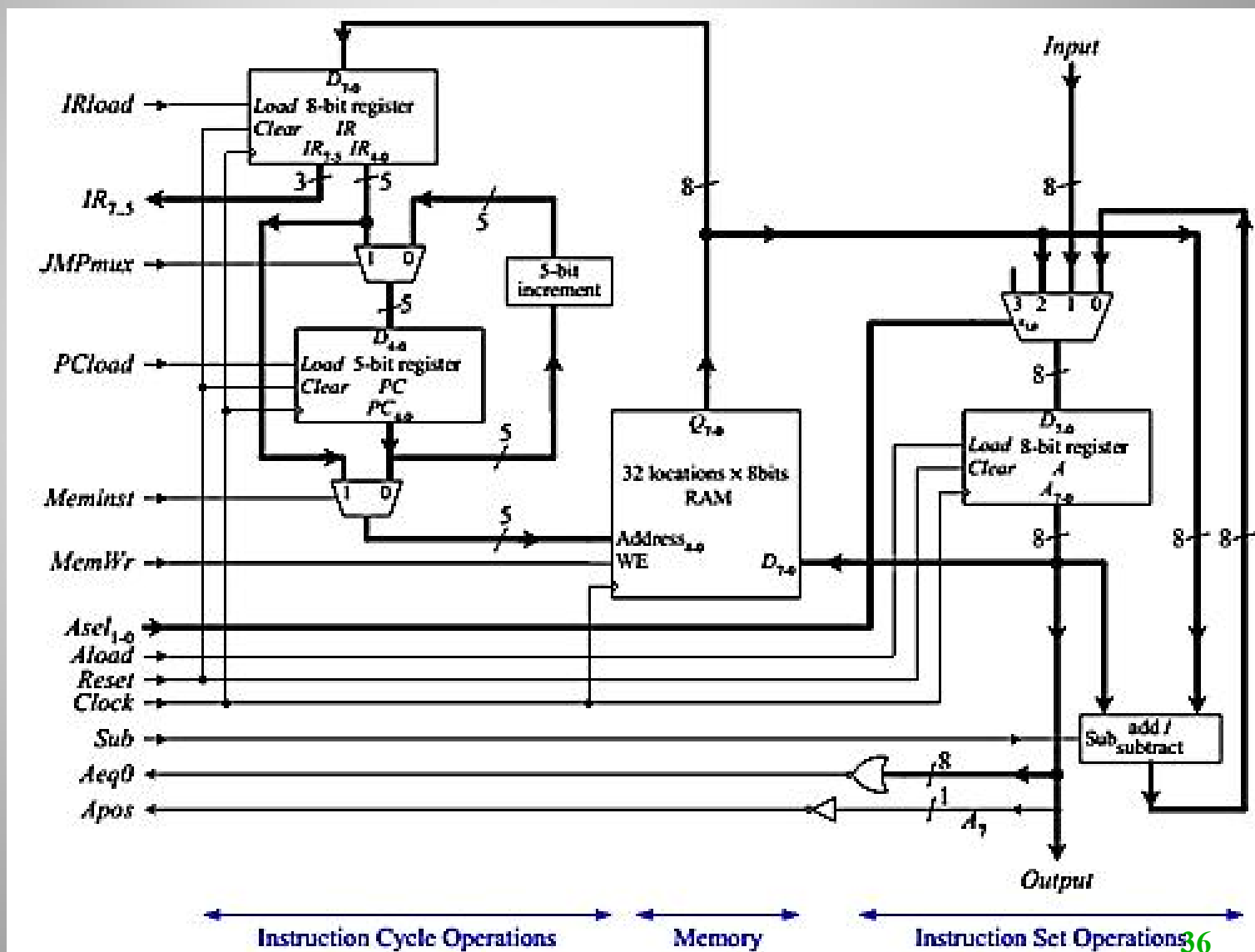
- LOAD指令
 - 把存储器特定地址中的内容赋给累加器A,
 - 地址是由指令的低5位表示。
- STORE指令
 - 把A中存储的内容赋给存储器特定地址单元。
- ADD和SUB指令
 - 分别表示把A的内容和存储器特定地址的内容相加、相减,
 - 并把其结果再赋给A。
- IN指令
 - 输入一个值并存入A。
- JZ指令
 - 如果A=0则把特定地址赋给PC。把新地址赋给PC会导致微处理器跳转到存储器这个新地址继续执行程序。
- JPOS
 - 表示如果A是正数则把A中的内容作为特定地址赋给PC。
 - 因为A是用补码表示, 所以如果A的最高位为0时, 则A是正数。
- HALT指令使得微处理器停止运行。

数据通路-2（自学）

- 执行指令周期操作和实现所有指令操作

PC和递增单元的位宽：需要有32个位置的存储器，因此地址、PC和递增单元的位宽均为5比特。

在PC的输出和存储器地址的输入之间的加入了一个2选1多路选择器。多路选择器的一个输入来自于PC，另一个输入则来自IR的低四位，IR4-0。



操作数

- 指令: LOAD、STORE、ADD、SUB
 - 多路选择器的一个输入来自于PC,
 - 另一个输入则来自IR的低5位, IR_{4-0} 。用地址作为操作数。
 - 它们的存储器地址是由IR的低五位给定的。
 - 2选1多路选择器的选通信号是Meminst。
- 存储器
 - 大小增加到32个位置, 因此需要5比特位宽
 - LOAD & STORE指令: 为了把数据从存储器中读出, 也能把A的值赋给特定地址存储器, 需要用一个RAM来代替之前的ROM。
 - 为了实现STORE操作, 把A的输出与存储器的数据输入 D_{7-0} 相连。
 - 当信号MemWr有效时, 则把A的值写入内存的指令地址。
 - 存储器的输出 Q_{7-0} 与累加器A的相连是为了执行LOAD的指令,
 - 通过4选1多路选择器
- 加-减单元
 - 为了执行ADD和SUB指令
 - 多路选择器的选通信号为 $Asel_{1-0}$, 第四个多路选择端没有使用

反馈信号

- 两个条件跳转指令JZ和JPOS
 - 数据通路提供给状态机两个反馈信号, Aeq0和Apos。
 - 如果A的值是0, 则Aeq信号输出为1, 故只需一个8输入NOR门。
 - 如果A的值是正数, 则Apos为1, 因为A是补码表示, 首位为0表示正数, 首位为1表示负数, 所以Apos只须等于A7的取反信号。
- 数据通路有八个控制信号
 - IRload, JMPmux, PCload, Meminst, MemWr, Ase11-0, Aload 和Sub
- 数据通路提供2个反馈信号
 - Aeq0 和 Apos。

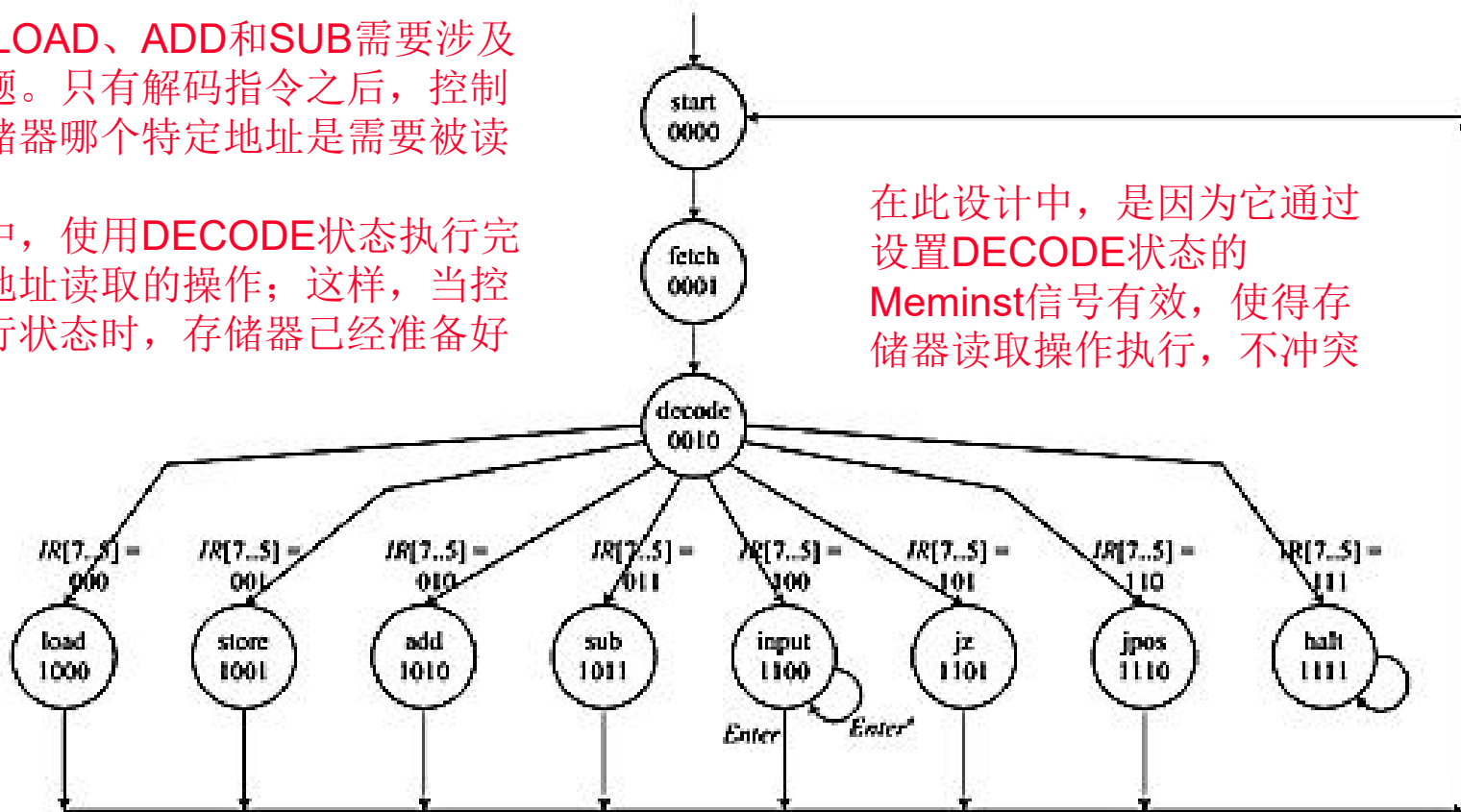
控制单元-2（自学）

• 状态图

- EC-2的DECODE状态需要通过分支到8个不同的状态以便执行对应的8个指令来解码8位指令编码

EC-2三个指令LOAD、ADD和SUB需要涉及存储器访问问题。只有解码指令之后，控制单元才知道存储器哪个特定地址是需要被读取的。

在我们的设计中，使用DECODE状态执行完成存储器特定地址读取的操作；这样，当控制单元到达执行状态时，存储器已经准备好了数据。



在此设计中，是因为它通过设置DECODE状态的Meminst信号有效，使得存储器读取操作执行，不冲突

控制信号

• 控制信号设置

在跳转到START状态前，状态表的INPUT状态将等待ENTER键信号。需要一个电路产生只持续一个时钟长度的有效按键信号

State $Q_3Q_2Q_1Q_0$	$IRload$	$JMPmux$	$PCload$	$Meminst$	$MemWr$	$Asel_{1,0}$	$Aload$	Sub	$Halt$
0000 start	0	0	0	0	0	00	0	0	0
0001 fetch	1	0	1	0	0	00	0	0	0
0010 decode	0	0	0	1	0	00	0	0	0
1000 load	0	0	0	0	0	10	1	0	0
1001 store	0	0	0	1	1	00	0	0	0
1010 add	0	0	0	0	0	00	1	0	0
1011 sub	0	0	0	0	0	00	1	1	0
1100 input	0	0	0	0	0	01	1	0	0
1101 jz	0	1	$Aeq0$	0	0	00	0	0	0
1110 jpos	0	1	$Apos$	0	0	00	0	0	0
1111 halt	0	0	0	0	0	00	0	0	1

(d)

• LOAD A指令

- $Asel_1$ 信号需要置1，而 $Asel_0$ 需要置0，为了多路选择器输入端口2使得存储器输出能到达A的输入。而A的实际加载是通过使 $Aload$ 信号置1实现的。

• 执行STORE A指令

- 需要通过使 $Meminst$ 置1使得存储器地址来自于IR，当 $MemWr$ 置1时，则将写入存储器。

状态转换

- 次态表和激励方程

Current State $Q_3Q_2Q_1Q_0$	Next State $Q_{3next} Q_{2next} Q_{1next} Q_{0next}$ (Implementation $D_3 D_2 D_1 D_0$)									
	IR_7, IR_6, IR_5								$Enter$	
	000 load	001 store	010 add	011 sub	100 input	101 jz	110 jpos	111 halt	0	1
0000 start	0001	0001	0001	0001	0001	0001	0001	0001		
0001 fetch	0010	0010	0010	0010	0010	0010	0010	0010		
0010 decode	1000	1001	1010	1011	1100	1101	1110	1111		
1000 load	0000	0000	0000	0000	0000	0000	0000	0000		
1001 store	0000	0000	0000	0000	0000	0000	0000	0000		
1010 add	0000	0000	0000	0000	0000	0000	0000	0000		
1011 sub	0000	0000	0000	0000	0000	0000	0000	0000		
1100 input									1100	0000
1101 jz	0000	0000	0000	0000	0000	0000	0000	0000		
1110 jpos	0000	0000	0000	0000	0000	0000	0000	0000		
1111 halt	1111	1111	1111	1111	1111	1111	1111	1111		

(b)

$$D_3 = Q_3'Q_2'Q_1Q_0' + Q_3Q_2Q_1'Q_0'Enter' + Q_3Q_2Q_1Q_0$$

$$D_2 = Q_3'Q_2'Q_1Q_0'IR_7 + Q_3Q_2Q_1'Q_0'Enter' + Q_3Q_2Q_1Q_0$$

$$D_1 = Q_3'Q_2'Q_1'Q_0 + Q_3'Q_2'Q_1Q_0'IR_6 + Q_3Q_2Q_1Q_0$$

$$D_0 = Q_3'Q_2'Q_1'Q_0' + Q_3'Q_2'Q_1Q_0'IR_5 + Q_3Q_2Q_1Q_0$$

(c)

控制信号

- 输出方程

$$IRload = Q_3'Q_2'Q_1'Q_0$$

$$JMPmux = Q_3Q_2Q_1'Q_0 + Q_3Q_2Q_1Q_0'$$

$$PCload = Q_3'Q_2'Q_1'Q_0 + Q_3Q_2Q_1'Q_0Aeq0 + Q_3Q_2Q_1Q_0'Apos$$

$$Meminst = Q_3'Q_2'Q_1Q_0' + Q_3Q_2'Q_1'Q_0$$

$$MemWr = Q_3Q_2'Q_1'Q_0$$

$$Asel_1 = Q_3Q_2'Q_1'Q_0'$$

$$Asel_0 = Q_3Q_2Q_1'Q_0'$$

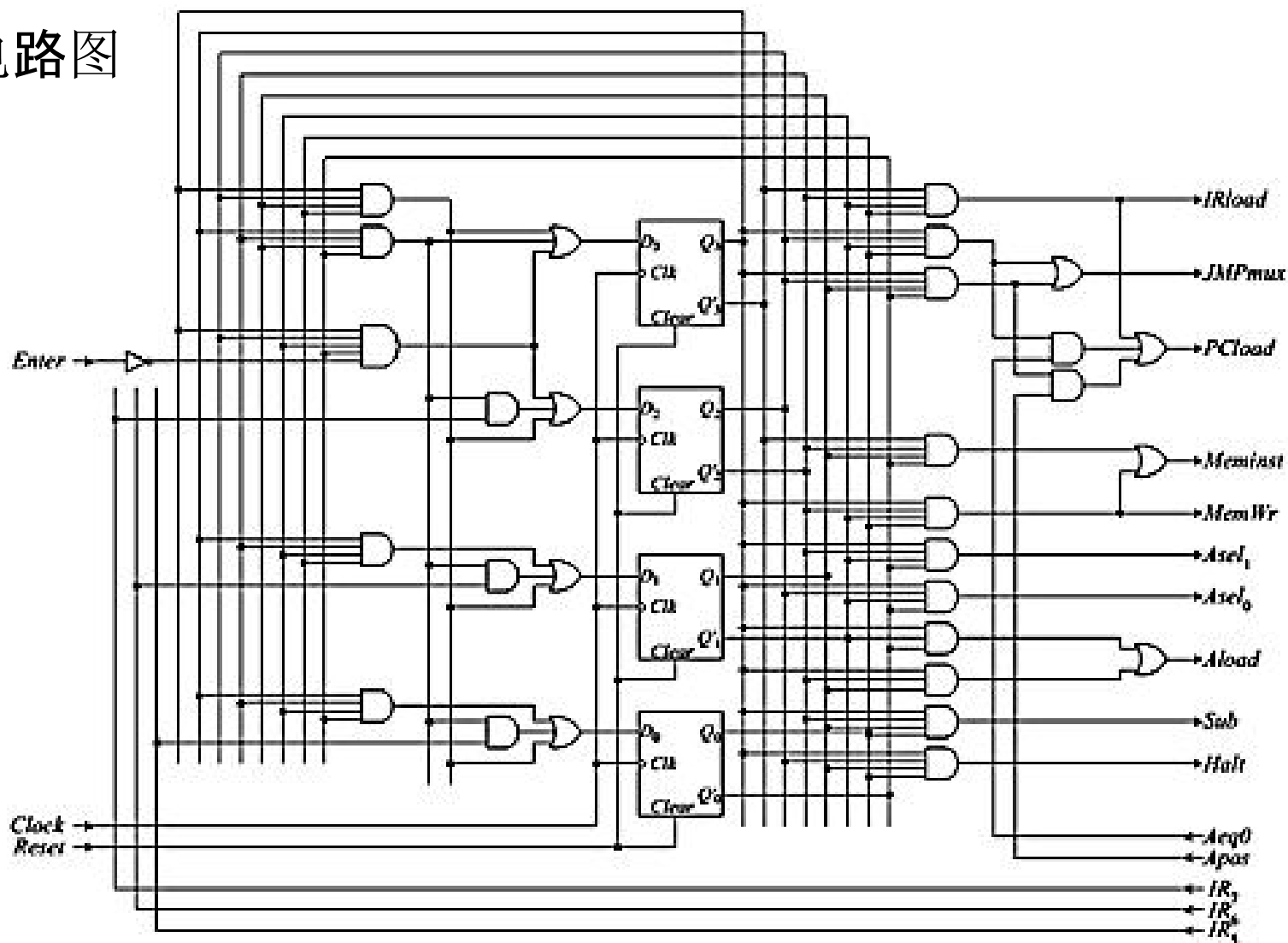
$$Aload = Q_3Q_1'Q_0' + Q_3Q_2'Q_1$$

$$Sub = Q_3Q_2'Q_1Q_0$$

$$Halt = Q_3Q_2Q_1Q_0$$

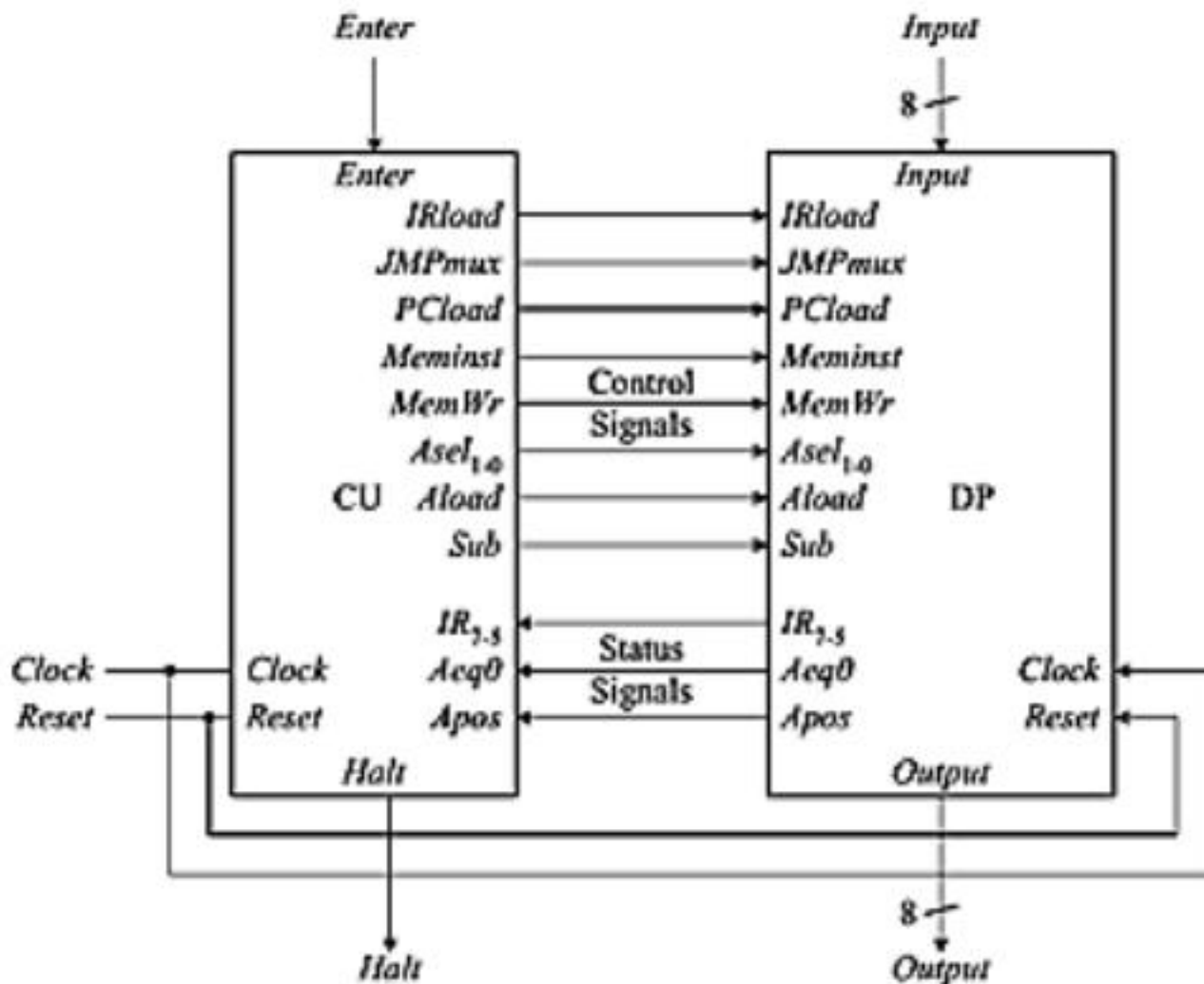
控制单元

- 电路图



完整电路-2（自学）

- 连接数据通路、控制电路和状态信号



示例程序-2

- 用文件PROGRAM.MIF的内容进行初始化
 - 文件包含3个程序：
 - GCD为计算两个输入的最大公约数的程序；
 - SUM计算1至n之间所有数的和的程序；
 - COUNT展示把输入n减至0的程序。

```
-- Content of the RAM memory in the file PROGRAM.MIF

DEPTH = 32; -- Depth of memory: 5-bit address
WIDTH = 8;  -- Width of memory: 8-bit data

ADDRESS_RADIX = BIN; -- All values in binary (HEX, DEC, OCT, BIN)
DATA_RADIX = BIN;

-- Opcodes for the EC-2
-- 000 = load A,aaaaa
-- 001 = store A,aaaaa
-- 010 = add A,aaaaa
-- 011 = sub A,aaaaa
-- 100 = in A
-- 101 = jz aaaaa
-- 110 = jpos aaaaa
-- 111 = halt

-- Specify the memory content.
```

```
-- Format of each memory location is
-- address : data

CONTENT
BEGIN
[00000..11111] : 00000000; -- Initialize locations range 00-1F to 0000
```

```
-----
-- There are three programs listed below: GCD, Sum, and Count
-- Only the program listed last is ran.
-- To try out a different program, move the code for the program you
-- want to the end of the list, re-compile, and download to the FPGA
```

```
-----
-- GCD
-- Program to calculate the GCD of two numbers
```

```
00000 : 10000000; -- input A
00001 : 00111110; -- store A,x
00010 : 10000000; -- input A
00011 : 00111111; -- store A,y

00100 : 00011110; -- loop: load A,x    -- x = y?
00101 : 01111111; -- sub A,y
00110 : 10110000; -- jz out            -- x=y
00111 : 11001100; -- jp xgty          -- x>y

01000 : 00011111; -- load A,y          -- y>x
01001 : 01111110; -- sub A,x           -- y-x
01010 : 00111111; -- store A,y
01011 : 11000100; -- jp loop

01100 : 00011110; -- xgty: load A,x    -- x>y
01101 : 01111111; -- sub A,y           -- x-y
01110 : 00111110; -- store A,x
01111 : 11000100; -- jp loop

10000 : 00011110; -- load A,x
10001 : 11111111; -- halt

11110 : 00000000; -- x
11111 : 00000000; -- y
```

```
-----
-- SUM
-- Program to sum N downto 1
```

```
00000 : 00011101; -- load A,one        -- zero sum by doing 1-1
00001 : 01111101; -- sub A,one
00010 : 00111110; -- store A,sum

00011 : 10000000; -- input A
00100 : 00111111; -- store A,n

00101 : 00011111; -- loop: load A,n    -- n + sum
00110 : 01011110; -- add A,sum
00111 : 00111110; -- store A,sum
```

```
01000 : 00011111; -- load A,n          -- decrement A
01001 : 01111101; -- sub A,one
01010 : 00111111; -- store A,n

01011 : 10101101; -- jz out
01100 : 11000101; -- jp loop
01101 : 00011110; -- out: load A,sum
01110 : 11111111; -- halt

11101 : 00000001; -- one
11110 : 00000000; -- sum
11111 : 00000000; -- n
```

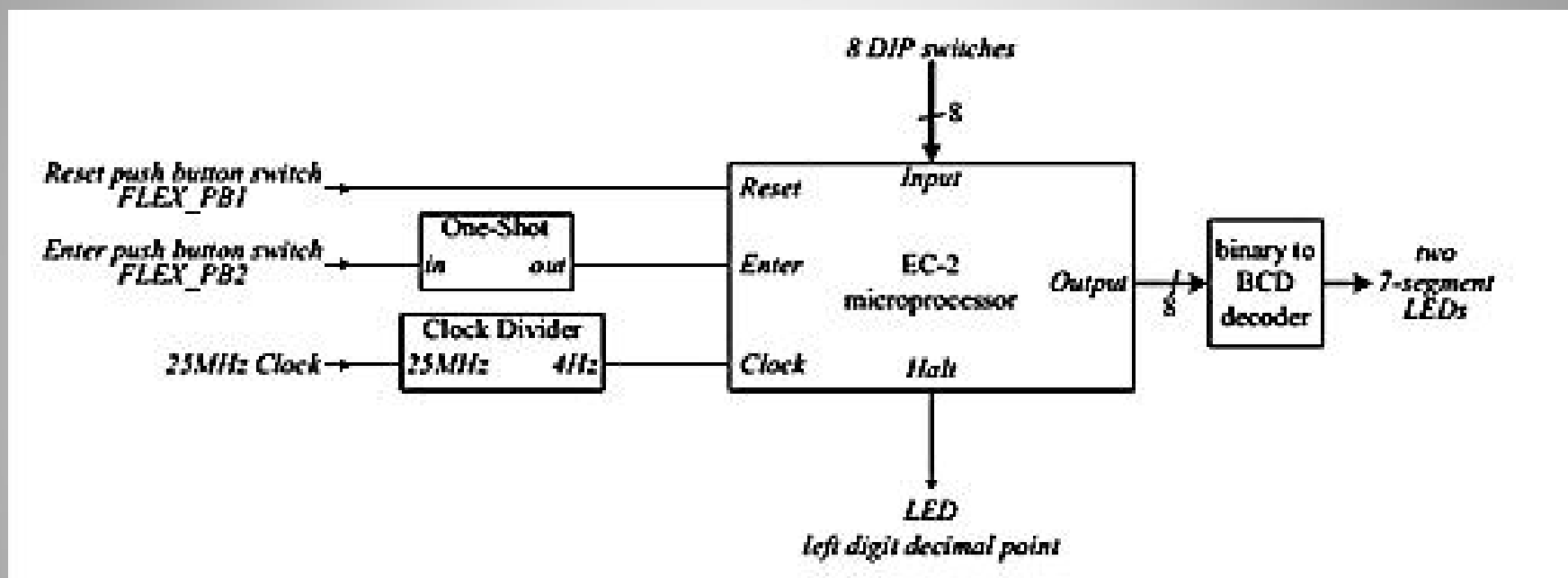
```
-----
-- COUNT
-- Program to countdown from input N to 0
```

```
00000 : 10000000; -- input
00001 : 01111111; -- Sub A,11111
00010 : 10100100; -- jz 00100
00011 : 11000001; -- jp 00001
00100 : 11111111; -- halt
11111 : 00000001; -- constant 1
```

```
END;
```

硬件实现-2（自学）

- EC-2微处理器的输入输出设备之间的接口



输入由8个DIP开关组成，输出是2个七段数码管显示。

使用一个LED来显示我们的微处理器是否已经停止工作，使用一个按钮作为复位按钮。

当Enter开关每次被按压时，一个单触发电路用来产生一个时钟周期的脉冲。

将系统输入的时钟进行分频可以看到一些中间结果的显示。

微处理器设计EC-2的Verilog实现（自学）

- 例p98/2019版
 - 2选1多路选择器
 - PC寄存器
 - Verilog代码综合出的电路
- CU(控制单元)代码p100-/2019版
- DP(数据通路)代码p103-
- 得到微处理器p105-/2019版
 - 将CU和DP模块连接起来Verilog代码

课后作业

- 1) 查阅:

- 国际电路公司的CPU、MCU、DSP:
 - » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....
- 图书馆资源: 电子器件天地, 软件
- <ftp://10.13.71.58>

- 2) 作业:

- 学在浙大

- 3) 阅读:

- 复习《数字系统设计与Verilog HDL》
- <http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/>

课后作业

- 4) Project

- Project2（选做平时成绩加分）：用具体逻辑电路描述设计并FPGA实现一个4bit简单CPU实物。（期末考试周停课前一周交）